

Data Sources: <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>

```
from google.colab import files
files.upload()
```



选择文件 kaggle.json

- kaggle.json(application/json) – 74 bytes, last modified: 2025/6/20 – 100% done
- Saving kaggle.json to kaggle.json

```
import os
os.makedirs("/root/.kaggle", exist_ok=True)
!mv kaggle.json /root/.kaggle/
!chmod 600 /root/.kaggle/kaggle.json
```

```
!kaggle datasets download -d mlg-ulb/creditcardfraud
```



Dataset URL: <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>
 License(s): DbCL-1.0
 Downloading creditcardfraud.zip to /content
 0% 0.00/66.0M [00:00<?, ?B/s]
 100% 66.0M/66.0M [00:00<00:00, 739MB/s]

```
import zipfile
with zipfile.ZipFile("/content/creditcardfraud.zip", "r") as creditcardfraud:
    creditcardfraud.extractall("creditcard-fraud-data")
```

```
import zipfile
zip_path = "creditcardfraud.zip"
with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    file_list = zip_ref.namelist()
    print("The type of ZIP profile: ")
    for file in file_list:
        print(file)
```



The type of ZIP profile:
 creditcard.csv

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.ensemble import RandomForestClassifier, IsolationForest
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score, roc_curve, precision_recall_curve, average_precision_score
from scipy.stats import multivariate_normal
from tabulate import tabulate
import io
import warnings
warnings.filterwarnings("ignore")

creditcardfraud = pd.read_csv("/content/creditcard-fraud-data/creditcard.csv")
creditcardfraud.head()
```



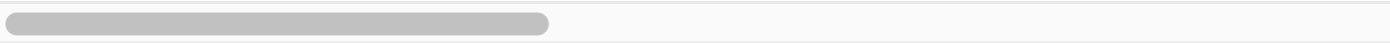
What can I help you build?





	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V21	
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787	...	-0.018307	0.2
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425	...	-0.225775	-0.6
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654	...	0.247998	0.7
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	0.377436	-1.387024	...	-0.108300	0.0
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	-0.270533	0.817739	...	-0.009431	0.7

5 rows × 31 columns



creditcardfraud.info()



```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Time        284807 non-null float64
1   V1          284807 non-null float64
2   V2          284807 non-null float64
3   V3          284807 non-null float64
4   V4          284807 non-null float64
5   V5          284807 non-null float64
6   V6          284807 non-null float64
7   V7          284807 non-null float64
8   V8          284807 non-null float64
9   V9          284807 non-null float64
10  V10         284807 non-null float64
11  V11         284807 non-null float64
12  V12         284807 non-null float64
13  V13         284807 non-null float64
14  V14         284807 non-null float64
15  V15         284807 non-null float64
16  V16         284807 non-null float64
17  V17         284807 non-null float64
18  V18         284807 non-null float64
19  V19         284807 non-null float64
20  V20         284807 non-null float64
21  V21         284807 non-null float64
22  V22         284807 non-null float64
23  V23         284807 non-null float64
24  V24         284807 non-null float64
25  V25         284807 non-null float64
26  V26         284807 non-null float64
27  V27         284807 non-null float64
28  V28         284807 non-null float64
29  Amount      284807 non-null float64
30  Class       284807 non-null int64
dtypes: float64(30), int64(1)
memory usage: 67.4 MB
```

creditcardfraud.describe()



	Time	V1	V2	V3	V4	V5	V6	V7	
count	284807.000000	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2
mean	94813.859575	1.168375e-15	3.416908e-16	-1.379537e-15	2.074095e-15	9.604066e-16	1.487313e-15	-5.556467e-16	
std	47488.145955	1.958696e+00	1.651309e+00	1.516255e+00	1.415869e+00	1.380247e+00	1.332271e+00	1.237094e+00	
min	0.000000	-5.640751e+01	-7.271573e+01	-4.832559e+01	-5.683171e+00	-1.137433e+02	-2.616051e+01	-4.355724e+01	-
25%	54201.500000	-9.203734e-01	-5.985499e-01	-8.903648e-01	-8.486401e-01	-6.915971e-01	-7.682956e-01	-5.540759e-01	
50%	84692.000000	1.810880e-02	6.548556e-02	1.798463e-01	-1.984653e-02	-5.433583e-02	-2.741871e-01	4.010308e-02	2
75%	139320.500000	1.315642e+00	8.037239e-01	1.027196e+00	7.433413e-01	6.119264e-01	3.985649e-01	5.704361e-01	3
max	172792.000000	2.454930e+00	2.205773e+01	9.382558e+00	1.687534e+01	3.480167e+01	7.330163e+01	1.205895e+02	:

8 rows x 31 columns

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
creditcardfraud["Amount"] = scaler.fit_transform(creditcardfraud["Amount"].values.reshape(-1, 1))
creditcardfraud["Time"] = scaler.fit_transform(creditcardfraud["Time"].values.reshape(-1, 1))

creditcardfraud["Class"].value_counts(normalize = True)
```



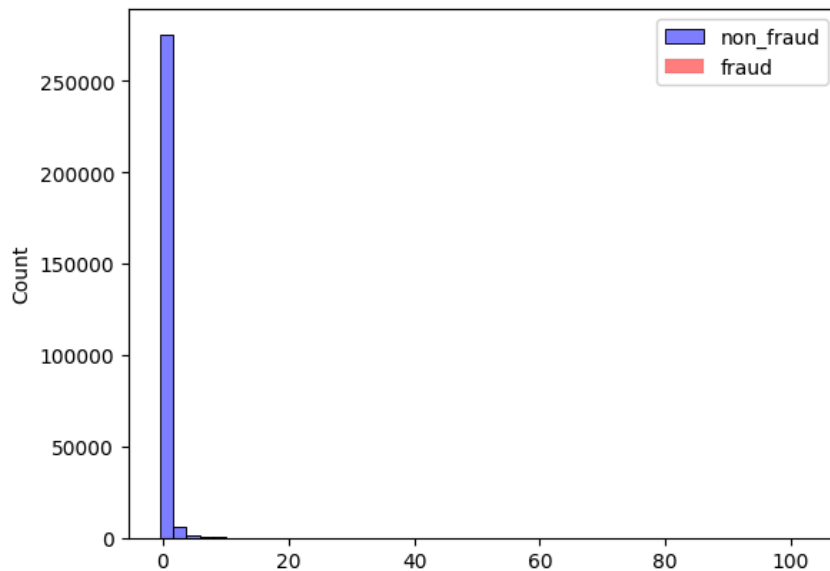
	proportion
Class	
0	0.998273
1	0.001727

```
creditcardfraud["Class"].value_counts()
```



	count
Class	
0	284315
1	492

```
non_fraud_amount = creditcardfraud.loc[creditcardfraud["Class"] == 0, "Amount"]
fraud_amount = creditcardfraud.loc[creditcardfraud["Class"] == 1, "Amount"]
sns.histplot(non_fraud_amount, bins=50, color="blue", label="non_fraud", alpha=0.5)
sns.histplot(fraud_amount, bins=50, color="red", label="fraud", alpha=0.5)
plt.legend()
plt.show()
```



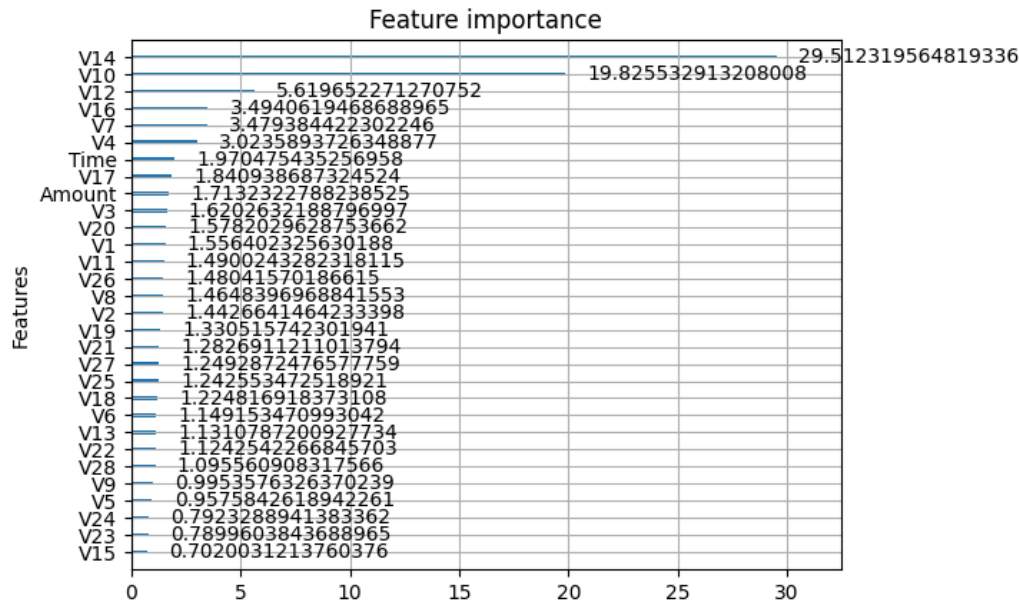
```
from imblearn.over_sampling import SMOTE
smote = SMOTE(random_state=42)
x = creditcardfraud.drop("Class",axis = 1)
y = creditcardfraud["Class"]
x_smote, y_smote = smote.fit_resample(x, y)
```

```
import xgboost as xgb
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report, roc_auc_score, roc_curve, auc
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
x_train, x_val, y_train, y_val = train_test_split(x_train, y_train, test_size=0.25, random_state=42)
model = xgb.XGBClassifier(max_depth=6, n_estimators = 100, use_label_encoder=False, eval_metric = "logloss")
model.fit(x_train, y_train)
```



```
XGBClassifier
XGBClassifier(base_score=None, booster=None, callbacks=None,
               colsample_bylevel=None, colsample_bynode=None,
               colsample_bytree=None, device=None, early_stopping_rounds=None,
               enable_categorical=False, eval_metric='logloss',
               feature_types=None, gamma=None, grow_policy=None,
               importance_type=None, interaction_constraints=None,
               learning_rate=None, max_bin=None, max_cat_threshold=None,
               max_cat_to_onehot=None, max_delta_step=None, max_depth=6,
               max_leaves=None, min_child_weight=None, missing=nan,
               monotone_constraints=None, multi_strategy=None, n_estimators=100,
               num_parallel_tree=None, random_state=None, ...)
```

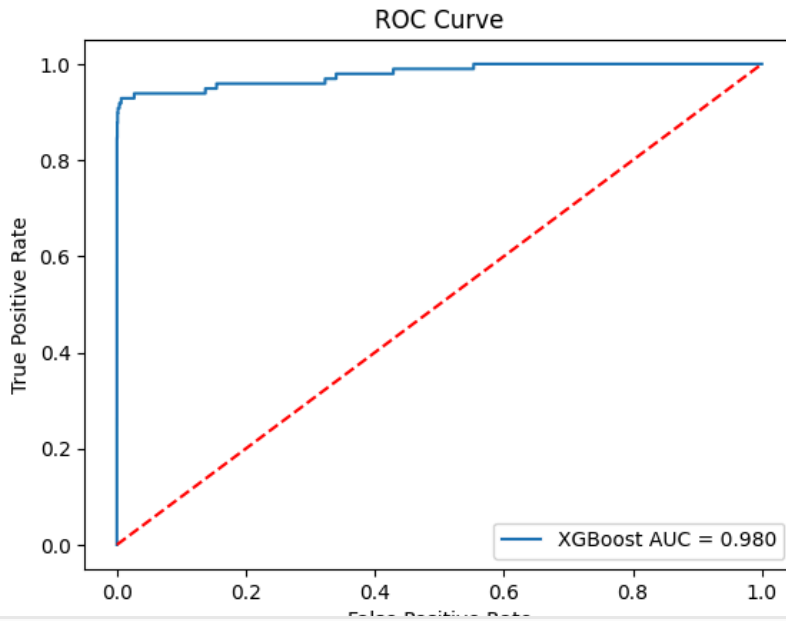
```
import matplotlib.pyplot as plt
xgb.plot_importance(model, max_num_features=31, importance_type='gain')
plt.show()
```



```

y_pred_proba = model.predict_proba(x_test)[: , 1]
fpr, tpr, _ = roc_curve(y_test, y_pred_proba)
plt.plot(fpr, tpr, label=f'XGBoost AUC = {roc_auc_score(y_test, y_pred_proba):.3f}')
plt.plot([0, 1], [0, 1], linestyle='--', color='red')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()

```



```

y_train_pred = model.predict_proba(x_train)[: , 1]
y_val_pred = model.predict_proba(x_val)[: , 1]

train_auc = roc_auc_score(y_train, y_train_pred)
val_auc = roc_auc_score(y_val, y_val_pred)

print(f"Train AUC: {train_auc:.4f}")
print(f"Validation AUC: {val_auc:.4f}")

```

```

↗ Train AUC: 1.0000
  Validation AUC: 0.9558

```

```

from sklearn.model_selection import RandomizedSearchCV
from xgboost import XGBClassifier

```

```

param_dist = {
    'max_depth': [3,5,7,9],
    'learning_rate': [0.01, 0.05, 0.1],
    'n_estimators': [100, 200, 300],
    'subsample': [0.6, 0.8, 1.0],
    'colsample_bytree': [0.6, 0.8, 1.0],
}

```

```

xgb = XGBClassifier(use_label_encoder=False, eval_metric='logloss')

```

```

search = RandomizedSearchCV(xgb, param_dist, n_iter=20, scoring='roc_auc', cv=5, verbose=2, random_state=42)
search.fit(x_train, y_train)

```

```

print("Best params:", search.best_params_)
print("Best AUC:", search.best_score_)

```

```

↗ [CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=3, n_estimators=100, subsample=0.6; total time= 2
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=3, n_estimators=100, subsample=0.6; total time= 4
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=3, n_estimators=100, subsample=0.6; total time= 2
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=3, n_estimators=100, subsample=0.6; total time= 2
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=9, n_estimators=300, subsample=1.0; total time= 10
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=9, n_estimators=300, subsample=1.0; total time= 10
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=9, n_estimators=300, subsample=1.0; total time= 9

```

```
Best params: {'subsample': 0.6, 'n_estimators': 100, 'max_depth': 3, 'learning_rate': 0.1, 'colsample_bytree': 0.8}
Best AUC: 0.9848738755168551
```

```
from xgboost import XGBClassifier
best_params = search.best_params_
best_model = XGBClassifier(**best_params, use_label_encoder=False, eval_metric='logloss')
best_model.fit(x_train, y_train)
```

```
XGBClassifier
XGBClassifier(base_score=None, booster=None, callbacks=None,
               colsample_bylevel=None, colsample_bynode=None,
               colsample_bytree=0.8, device=None, early_stopping_rounds=None,
               enable_categorical=False, eval_metric='logloss',
               feature_types=None, gamma=None, grow_policy=None,
               importance_type=None, interaction_constraints=None,
               learning_rate=0.1, max_bin=None, max_cat_threshold=None,
               max_cat_to_onehot=None, max_delta_step=None, max_depth=3,
               max_leaves=None, min_child_weight=None, missing=nan,
               monotone_constraints=None, multi_strategy=None, n_estimators=100,
               n_jobs=None, num_parallel_tree=None, random_state=None, ...)
```

```
from sklearn.metrics import roc_auc_score, classification_report
```

```
y_val_pred_proba = best_model.predict_proba(x_val)[: , 1]
y_val_pred = best_model.predict(x_val)
print("Final AUC on validation set:", roc_auc_score(y_val, y_val_pred_proba))
print(classification_report(y_val, y_val_pred))
```

```
Final AUC on validation set: 0.9620870363258381
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	56884
1	0.94	0.79	0.86	78
accuracy			1.00	56962
macro avg	0.97	0.90	0.93	56962
weighted avg	1.00	1.00	1.00	56962

```
from sklearn.model_selection import cross_val_score
xgb = XGBClassifier(**search.best_params_, use_label_encoder=False, eval_metric='logloss')
scores = cross_val_score(xgb, x, y, cv=5, scoring='roc_auc')
print("CV AUC scores:", scores)
print("Mean CV AUC:", scores.mean())
```

```
CV AUC scores: [0.99177804 0.97110501 0.96115108 0.99313764 0.98431362]
Mean CV AUC: 0.9802970792602513
```

```
import numpy as np
x_noisy = x.copy()
x_noisy += np.random.normal(0, 0.1, x_noisy.shape)
auc_noisy = roc_auc_score(y, model.predict_proba(x_noisy)[: , 1])
print(f"AUC with noise added: {auc_noisy:.4f}")
```

```
AUC with noise added: 0.9909
```

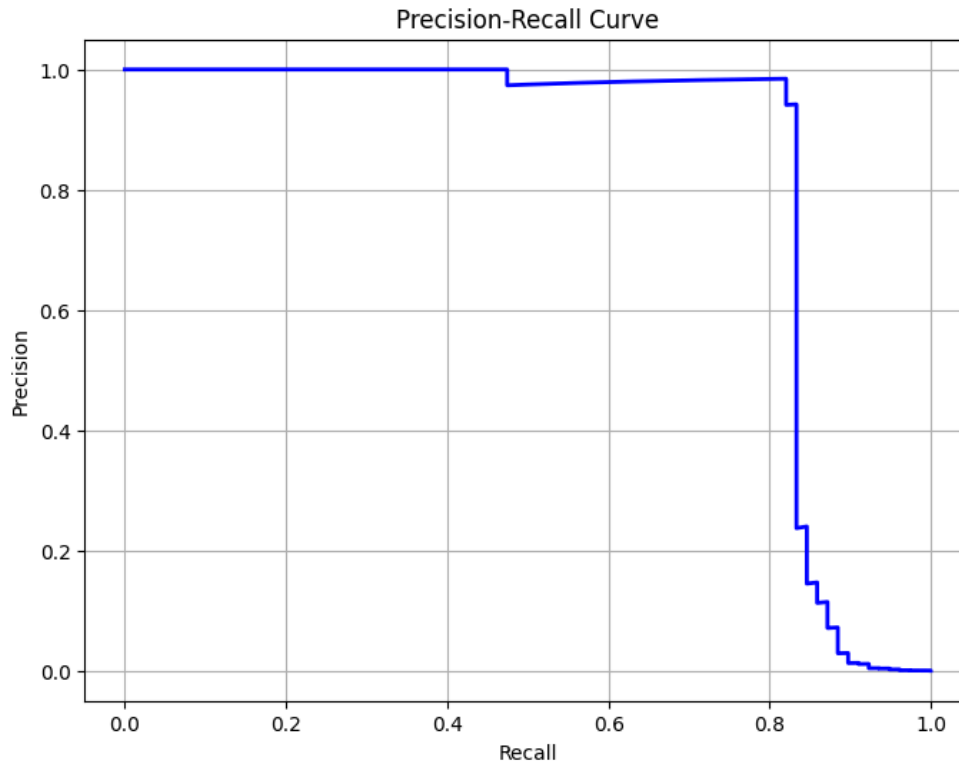
```
from sklearn.metrics import precision_recall_curve
y_scores = model.predict_proba(x_val)[: , 1]
precision, recall, thresholds = precision_recall_curve(y_val, y_scores)
idx = np.where(recall > 0.8)[0][-1]
print(f"Threshold: {thresholds[idx]:.4f}, Precision: {precision[idx]:.4f}, Recall: {recall[idx]:.4f}")
```

```
Threshold: 0.5101, Precision: 0.9844, Recall: 0.8077
```

```
import matplotlib.pyplot as plt
from sklearn.metrics import precision_recall_curve
```

```
y_scores = model.predict_proba(x_val)[: , 1]
precision, recall, _ = precision_recall_curve(y_val, y_scores)
```

```
plt.figure(figsize=(8,6))
plt.plot(recall, precision, color='b', lw=2)
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve')
plt.grid(True)
plt.show()
```



```
import joblib
joblib.dump(best_model, 'best_model.pkl')
```



```
['best_model.pkl']
```

```
from sklearn.ensemble import IsolationForest
isoforest = IsolationForest(n_estimators=100, contamination=0.001, random_state=42)
isoforest.fit(x)
outlier_preds = isoforest.predict(x)
is_anomaly = (outlier_preds == -1)
```

#输出异常样本的统计

```
import pandas as pd
x_with_outliers = x.copy()
x_with_outliers['is_anomaly'] = is_anomaly
x_with_outliers['Class'] = y.values
print(x_with_outliers[x_with_outliers['is_anomaly'] == True]['Class'].value_counts(normalize=True))
```



```
Class
0    0.708772
1    0.291228
Name: proportion, dtype: float64
```

```
anomaly_score = isoforest.decision_function(x) * -1
x_augmented = x.copy()
x_augmented['anomaly_score'] = anomaly_score
```



```
!pip install scorecardpy
import scorecardpy as sc
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report, roc_auc_score, roc_curve, auc
from sklearn.preprocessing import StandardScaler
x = creditcardfraud.drop(columns='Class')
y = creditcardfraud['Class']
data = pd.concat([x, y], axis=1)
bins = sc.woebin(data. v='Class'. method='tree')
```